

IBM Z and the z/OS Operating System

Overview





DEMYSTIFYING THE MAINFRAME



Most people in the software field feel comfortable working with the mainstream operating systems we encounter every day – Unix, Linux, Android, OS X, iOS, Microsoft Windows, and a few others that are all broadly similar.

But the IBM Z and its traditional operating systems – z/OS, z/VSE, and z/TPF – seem to come from another world.

[Cue spooky music]

Do we dare open this enigmatic box? If we do, what mysteries await within?

Well, we *have* to open it if we intend to complete this bootcamp. So, let's get to it.

DEMYSTIFYING THE MAINFRAME

On the most basic level, the IBM Z is based on the same conceptual architecture as all the other computers we've worked on. It's a von Neumann machine.

We take it for granted today, but in 1945 John von Neumann's conceptual diagram for a practical digital computer set the stage for decades of rapid technological development.

The mainframes, minicomputers, and microcomputers developed from the 1950s onward are all based on the same general model. The hardware has evolved considerably, but they share the components laid out in von Neumann's diagram. Their operating systems have to control the same general types of functions.

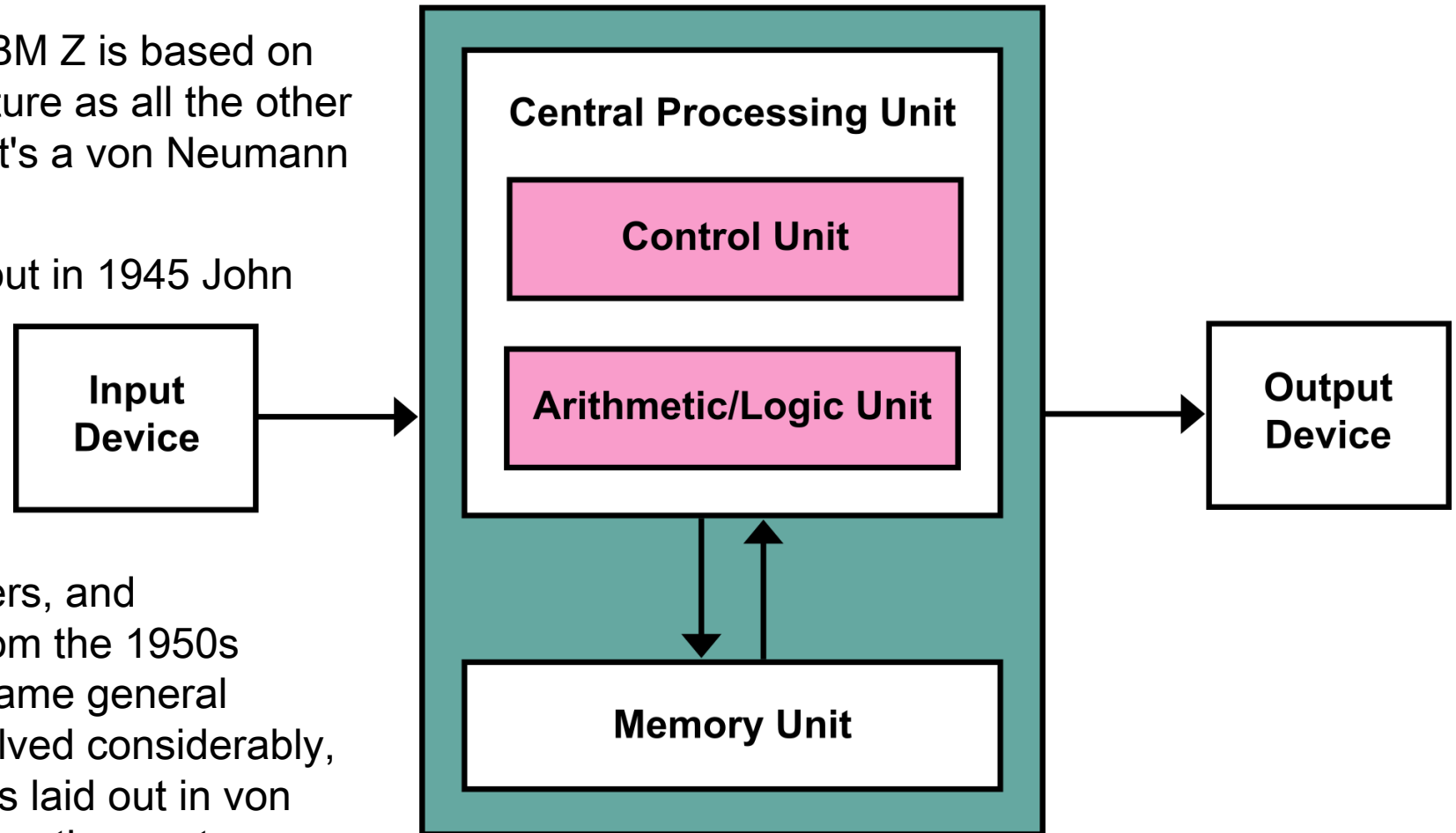


Image: Wikipedia. License: CC BY-SA 3.0

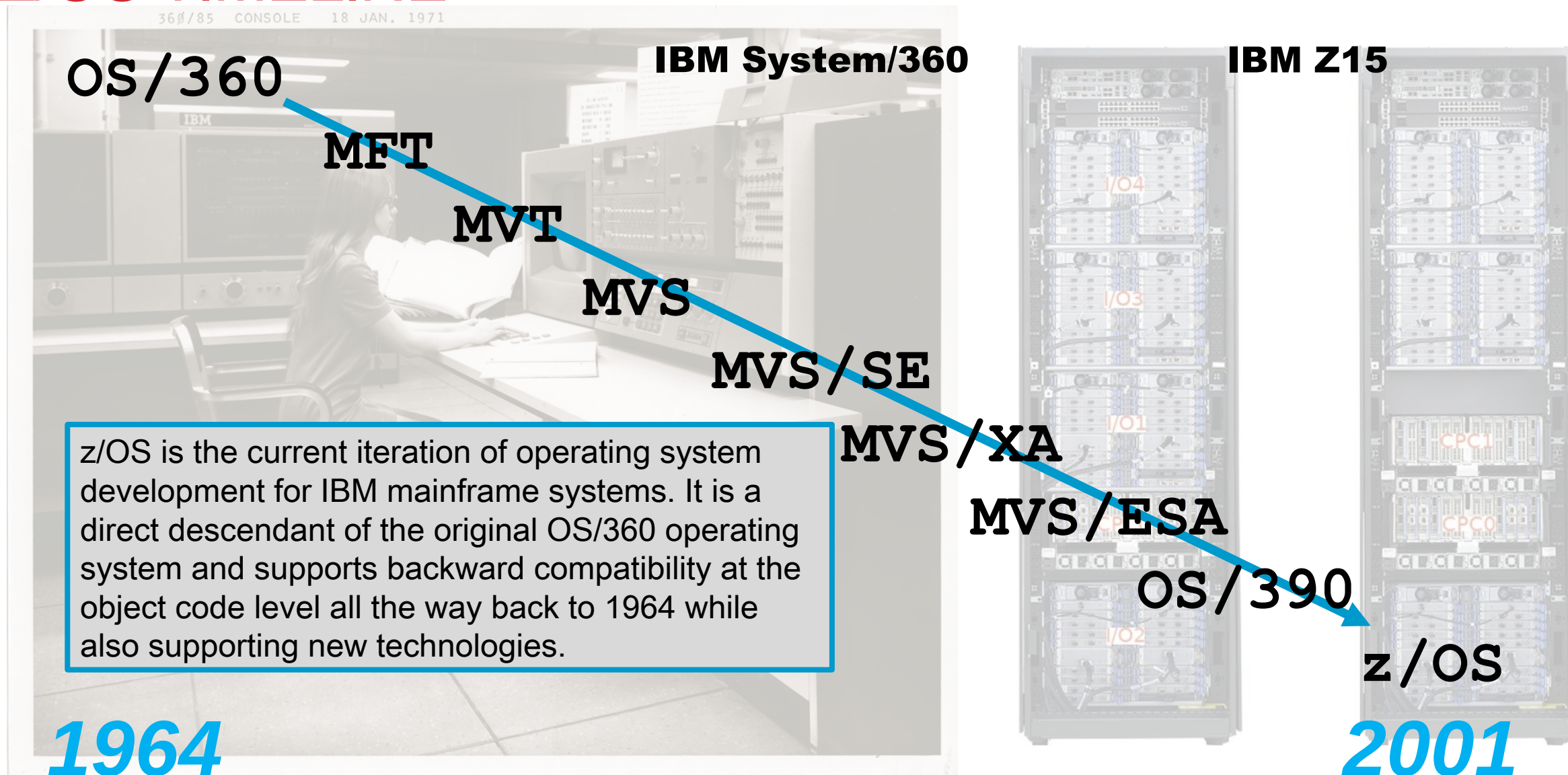
DEMYSTIFYING THE MAINFRAME



Various implementations of von Neumann's model are possible, and many have been built. The implementation details of the IBM Z and the z/OS operating system are certainly different from those of more-familiar platforms, but mainframes solve the same kinds of problems as other systems and you will find many familiar elements here.

Some of the differences boil down to terms – *virtual storage* instead of *memory*; *direct access storage device* instead of *hard disk drive*; *data set* instead of *file*. The system seems pretty mysterious until you get used to the terminology. But there are also more-interesting differences, such as the absence of a filesystem.

Z/OS TIMELINE



CORRELATING UNIX AND Z/OS CONCEPTS & TERMS

Assuming you're familiar with Unix and Linux, it might be helpful to map basic concepts between those systems and z/OS as a shortcut for picking up z/OS jargon. These items are conceptually equivalent even if they are implemented quite differently. Don't worry if the details are unfamiliar at this point.

Unix

z/OS

Boot the system

IPL the system. IPL stands for Initial Program Load.

User space

Problem state. A mode of execution rather than a memory area.

Kernel space

Supervisor state. A mode of execution rather than a memory area.

/etc filesystem

PARMLIB. Shipped as SYS1.PARMLIB, typically copied and customized.

systemd service manager

Primary subsystem. Always the Job Entry Subsystem (JES2 or JES3).

services

Secondary subsystems.

HDD, SSD, etc.

DASD. Direct Access Storage Device.

filesystem

Access methods. DASD is not preformatted with a filesystem.

process

Job. z/OS is fundamentally a batch processing system.

thread

Task or Service Request Block (SRB).

stdin, stdout, stderr

SYSIN (punched card images), SYSOUT and SYSERR (print line images).

console

TSO (Time-Sharing Option).

main memory

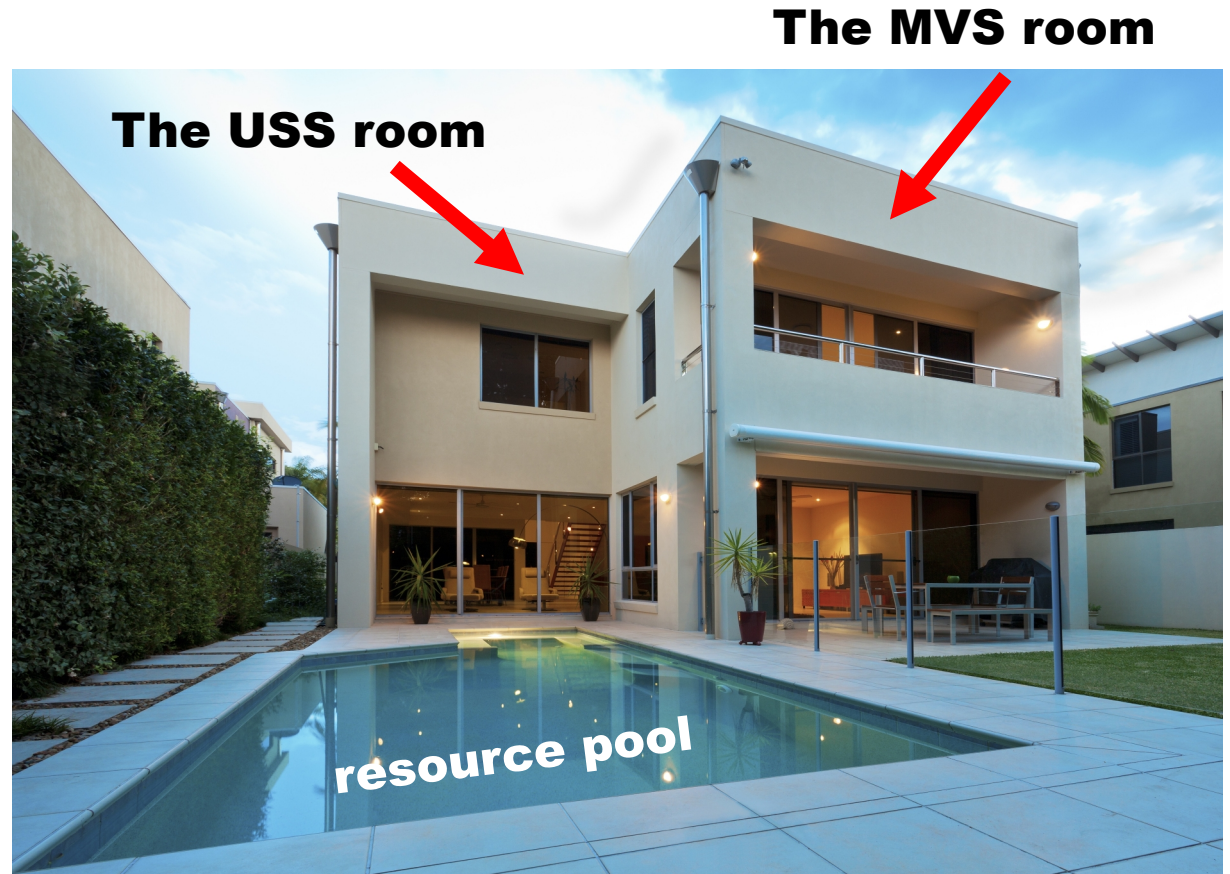
Virtual storage.

TWO SIDES OF THE SYSTEM

z/OS can be hard to describe because it contains so many different facilities. Since the 1990s, the system has supported POSIX functionality. It supports both the traditional MVS-style resources and features and Unix-like POSIX resources and features. The latter are branded as Unix System Services (USS).

People often refer to the "Unix side" and the "MVS side" of the system, but there is no Unix instance running under z/OS. That is only a convention to simplify conversations. Both sets of features are integral to z/OS.

It's as if you looked into a house through two different windows. One window affords a view of **the USS room**, furnished with filesystems and directories and a command line. The other other affords a view of **the MVS room**, furnished with access methods and qualified data set names and TSO. But it's the same house, and you can see through to the other room.



DIRECT ACCESS STORAGE DEVICE



By I, Deep silence, CC BY 2.5,
<https://commons.wikimedia.org/w/index.php?curid=2289786>

For purposes of backward compatibility, z/OS sees all external storage devices as DASD ("dazz-dee"). These were magnetic disk drives comprising stacks of platters with read/write heads mounted on arms that moved in and out between the platters. One circle around a platter is a *track*. All the tracks that are vertically aligned in the stack of platters is a *cylinder*.

While the physical technology is long gone, we still refer to external storage space in terms of tracks and cylinders.

The devices are virtual, but z/OS doesn't know that.

L: IBM 2311 drive, 1964.

R: Guts of a drive after a head crash.



By Vanderdecken - Own work, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=20599971>

HOW Z/OS MANAGES FILES (DATA SETS)

On the MVS "side" of z/OS, *files* were originally called *data sets*. Today, people use the two terms interchangeably.

Systems you've worked with before almost certainly had filesystems, and files were almost certainly organized under *directories* or *folders*. It's the same on the USS "side" of z/OS, but MVS did not have filesystems or directories.

Instead, a data set name comprises one or more segments of between one and eight characters each, separated by periods, with a total length not to exceed 44 characters.

For data set names containing more than one segment (practically all of them), the last segment is the data set name and the preceding segments are called qualifiers. (We refer to the whole thing as the data set name, however.) The first segment is the high-level qualifier. You'll see HLQ in the documentation. Any segments between the HLQ and the name are called mid-level qualifiers (MLQs). The qualifiers are used to control access rights.

USS

path
↓

`/home/smithj/projects/bootcamp/notes.txt`

filename *extension
or suffix*
↓ ↓

MVS

HLQ
↓

MLQ
↓

*filename
or library*
↓

member
↓

`MTR461.BOOTCAMP.CAPSTN(NOTES)`

HOW Z/OS MANAGES FILES (DATA SETS)

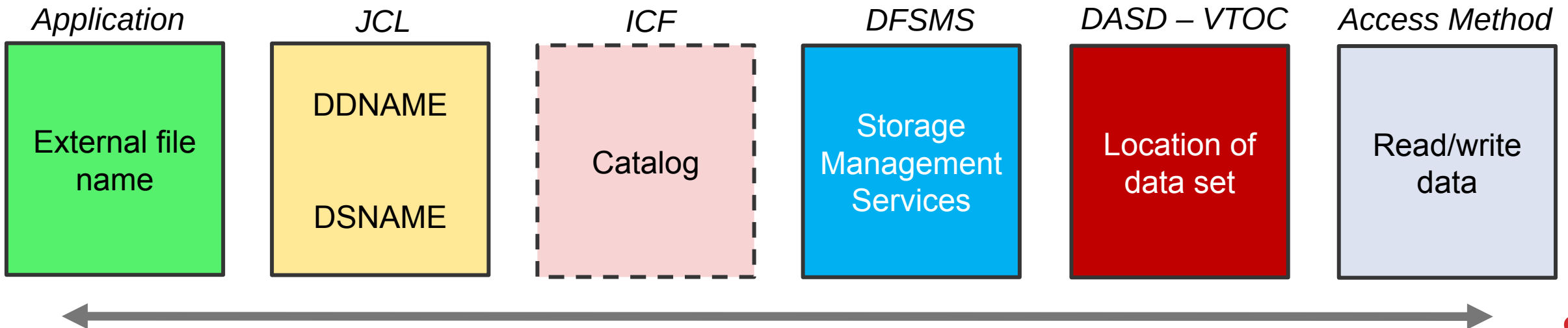
Each DASD unit is called a *volume*. While volumes are not formatted with a filesystem, they do have to be initialized with a *volume table of contents*, or VTOC ("vee-tock").

Data sets may be catalogued or uncatalogued. The z/OS functionality to manage catalogs is called the Integrated Catalog Facility (ICF). Catalogs store information about a data set's location as well as a few other attributes. You can refer to a catalogued data set just by its name, and the system will find it.

You can think of *catalogs* as analogous to *inode tables*, but bear in mind this is a high-level conceptual correlation and the implementations are completely different.

To refer to a data set that is not catalogued, you must specify the volume(s) where it resides. The system will then locate the data by consulting the VTOC.

Data storage is managed by the Data Facility Storage Management Subsystem (DFSMS) of z/OS.



TYPES OF DATA SETS

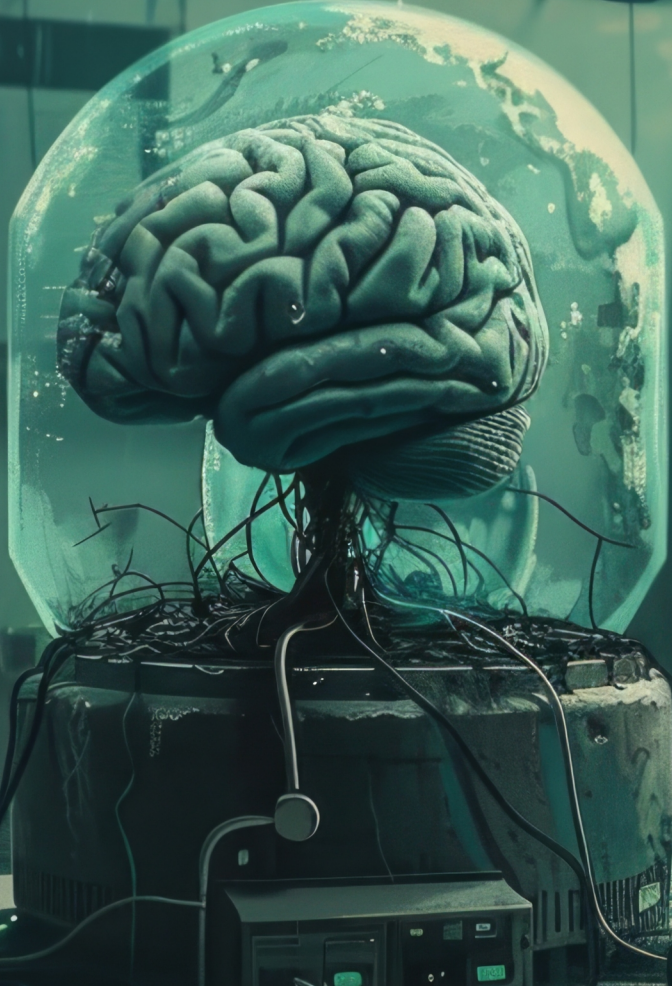
z/OS supports several types of data sets. Each type is managed by an *access method*. An access method is a set of functions that knows the format and structure of a particular type of data set and that exposes APIs to enable applications to work with that type of data set.

Supported data access

Access Method	Sequential	Direct	Indexed	Skip-Sequential	Partitioned	Byte Stream
BSAM	Y					
QSAM	Y					
BDAM		Y				
BPAM	Y				Y	
VSAM ESDS	Y					
VSAM RRDS	Y	Y		Y		
VSAM KSDS	Y		Y	Y		
VSAM LDS						Y

VIRTUALIZATION - BRAIN IN A JAR

Deep within the chambers of z/OS's heart, the system still expects input to look like 80-column punched cards, system output to look like 132-character print lines, and external storage devices to look like IBM 3390 DASD units. That's for the sake of backward compatibility all the way to 1964. But those things don't exist anymore. So, how does z/OS support new technologies?



Imagine a human brain removed from the body and installed in a jar. It can be connected to any and all kinds of sensory and motor devices, far beyond the capabilities of a living human. But it only has circuitry to react to the five senses and control a standard human body. All sensory input and all output signals must be converted between forms that a human brain can interpret and forms that the external devices can interpret.

z/OS is something like a brain in a jar. It can work with any modern technology, but at some point the signals have to be translated into forms the original brain understands. This is the reason you will see constraints such as 80-byte fixed-length records, or external storage defined in terms of tracks and cylinders.

WHAT YOU KNOW AND WHAT YOU CAN DO

- You understand that z/OS has many structural and functional features in common with other operating systems, even if their implementation is different.
- You have a sense of the history/evolution of z/OS from the System/360 to the IBM Z.
- You know z/OS includes functionality to support traditional mainframe operations as well as POSIX support for newer applications.
- You have a general idea of how concepts and terminology map between Unix and z/OS.
- You have a general idea of how z/OS manages files and the types of files it supports.

WHAT'S NEXT

- Systems Programming roles and facilities